

1 Abstract

This report details the quality assessment of pre-injection baseline data from 11 High-Performance Liquid Chromatography (HPLC) runs. The primary objective was to verify column integrity, segment baseline regions, and quantify signal stability using statistical metrics and visual analysis. Critical findings include the identification of a mislabeled data column, a significant metadata anomaly regarding sample identifiers, and a systemic violation of noise thresholds across all runs. While no runs violated physical detector limits, Run 32 exhibited extreme instability, and all runs demonstrated non-zero drift slopes requiring correction. The analysis confirms that raw data contains substantial noise and non-linear drift components, necessitating the application of advanced baseline correction models prior to quantitative integration.

2 Introduction

Accurate quantitative analysis in HPLC relies heavily on the stability of the pre-injection baseline. Systematic artifacts such as constant offsets, linear drift, and non-linear trends can significantly compromise peak integration and subsequent concentration calculations. This study focuses on evaluating the raw sensor signals (UV_1.280_mAU and UV_2.260_mAU) within the pre-injection region (volume ≤ 0 mL) across 11 distinct chromatography runs. The motivation for this analysis is to validate the equilibrium stability of the detector, identify data integrity issues such as mislabeled columns or metadata inconsistencies, and determine if the observed noise and drift exceed regulatory standards (USP $\leq 0.5\%$) or physical detector limits. By quantifying these artifacts, we aim to establish robust rejection criteria and define appropriate correction models, such as linear or polynomial fitting, to ensure data integrity before downstream processing.

3 Methodology

The data analysis workflow comprised three sequential steps. First, Column Integrity Verification and Baseline Data Segmentation were performed on the combined dataset. This involved calculating the correlation between 'UV_1.280_ml' and 'volume_ml' to identify mislabeled duplicates and filtering rows where 'volume_ml' ≤ 0 to isolate the baseline region. Metadata consistency was validated by checking the missingness rates of 'Sample.Code' and 'Fraction.unique_ID'. Second, Run-Specific Baseline Stability & Drift Profiling was conducted for each of the 11 runs. Linear regression was applied to the baseline data to calculate drift slopes and signal offsets, while Residual Standard Deviation (RSD) was computed to assess noise floors. A second-order polynomial fit was also generated to detect non-linear trends, comparing the R^2 improvement against the linear model. Finally, Visual Baseline Trend Analysis was executed for the first five runs to provide graphical evidence of the statistical findings, overlaying raw data with linear and polynomial fits and highlighting physical limit violations.

4 Results and Discussion

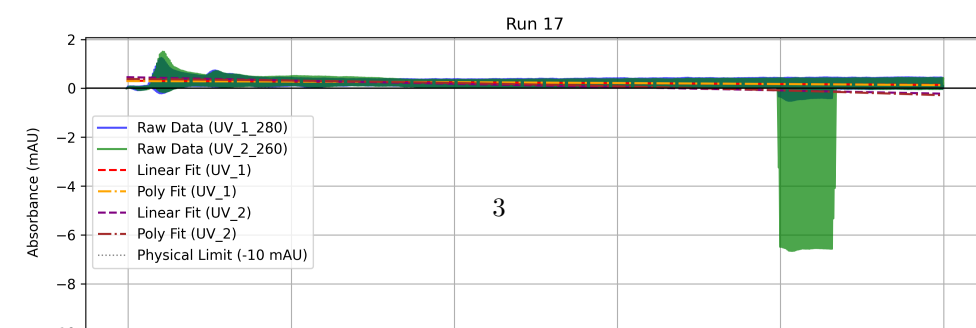
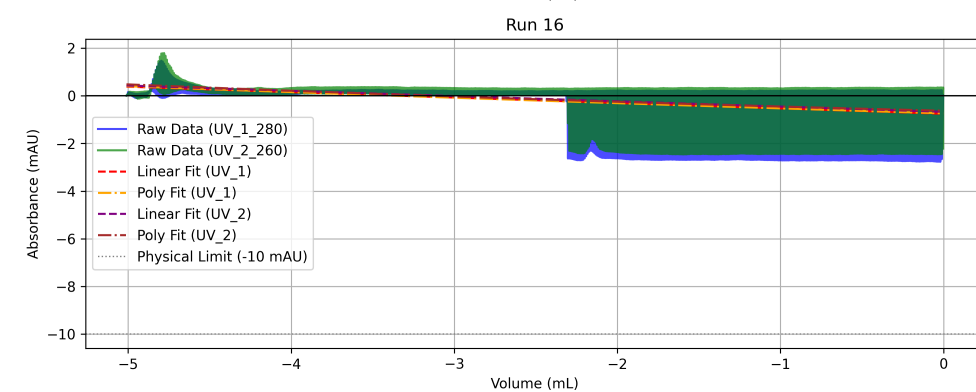
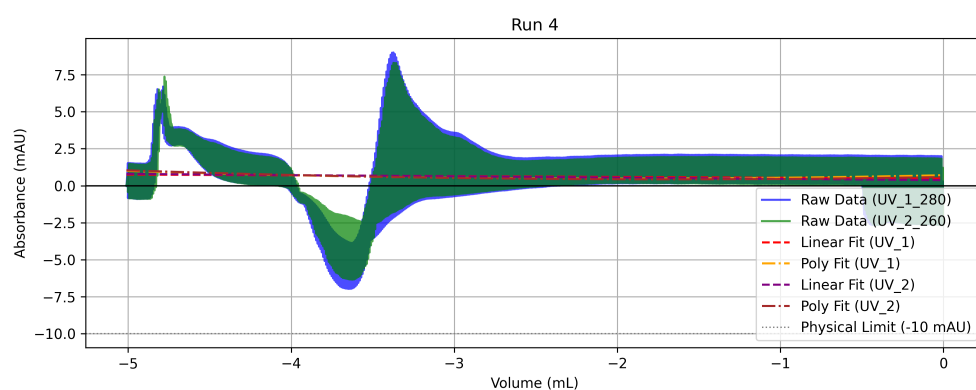
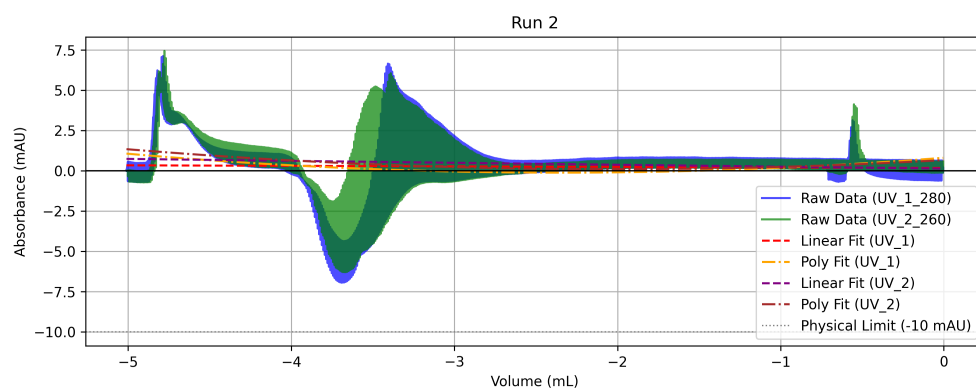
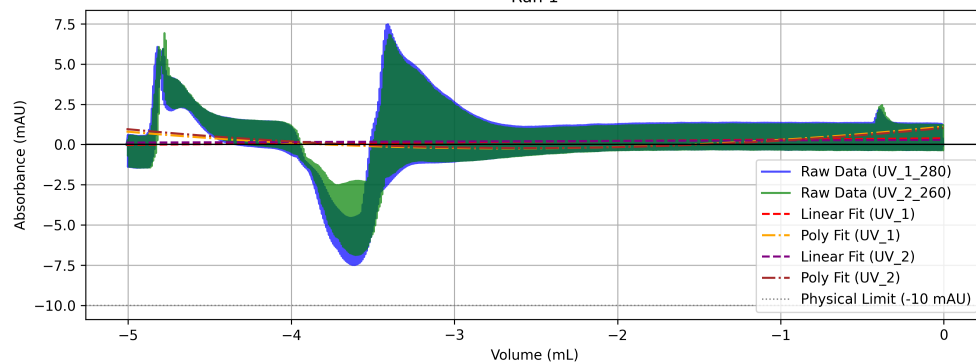
The initial data integrity check revealed a critical structural error: the column 'UV_1.280_ml' showed a perfect correlation (1.0000) with 'volume_ml', confirming it as a mislabeled duplicate of the volume axis rather than a UV signal. Consequently, this column was excluded from all signal analyses. While the baseline segmentation successfully identified between 1,804 and 6,360 rows per run, a significant metadata anomaly was detected. The analysis expected approximately 75% missingness in sample identifiers for the baseline region; however, the observed missingness was 99.99%. This deviation suggests a systemic data entry error or an overly broad segmentation logic, preventing immediate validation of the baseline segment as intended.

Regarding signal stability, the analysis of the 11 runs revealed a systemic issue with noise levels. Every run exhibited a 'True' status for Noise_Threshold_Violation, indicating that the baseline noise (RSD) exceeded the regulatory threshold (likely $\leq 0.5\%$ per USP $\leq 0.5\%$). Run 32 emerged as a severe outlier, displaying extreme instability with UV_260 RSD of 819.47 and UV_280 RSD of 42.05, suggesting potential sensor malfunction or data corruption. In contrast, Run 18 demonstrated the most stable baseline with low RSD

values (0.41 and 0.56). Despite the high noise, no runs violated physical detector limits (signal ≥ -10 mAU), although negative offsets were observed (e.g., -0.75 mAU in Run 16).

As illustrated in Figure 1, the visual analysis of the first five runs confirms the statistical findings of systematic drift. The raw data exhibits clear non-zero slopes, indicating linear drift across the baseline period. Furthermore, the second-order polynomial fits track the curvature of the raw data more closely than the linear models in several instances, suggesting the presence of non-linear drift components. The visualization also highlights regions where signal values approach the -10 mAU physical limit, validating the necessity of the proposed baseline correction protocols. The combination of high noise floors, systematic drift, and non-linear trends confirms that raw data cannot be used for quantitative integration without applying run-specific correction models, particularly for the highly unstable Run 32 which requires immediate investigation or exclusion.

Pre-Injection Baseline Drift & Non-Linearity by Run (First 5 Runs)

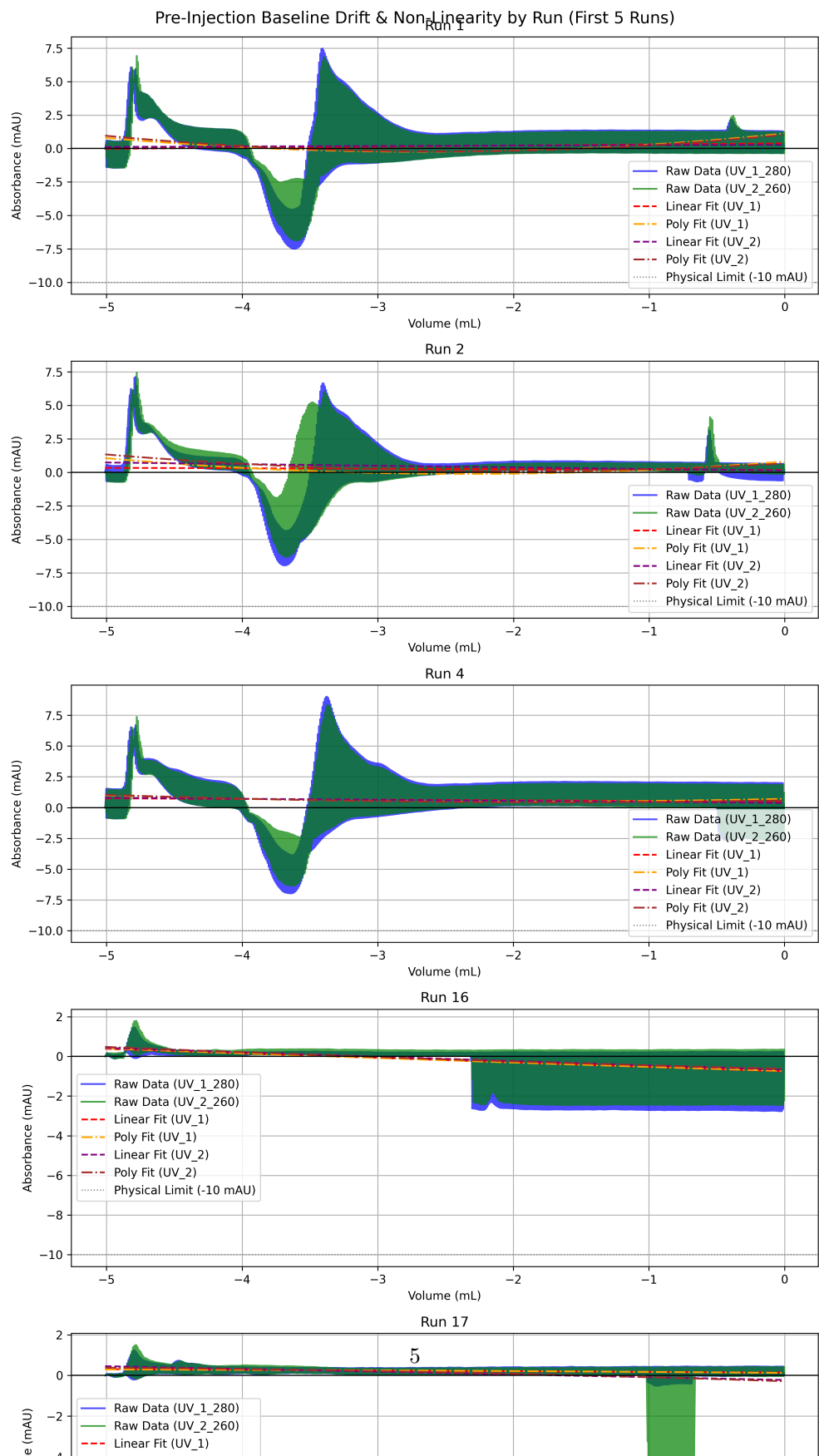


5 Conclusion

The analysis of the HPLC pre-injection baseline data has identified significant data quality issues that must be addressed before quantitative analysis. A mislabeled column was successfully identified and removed, but a pervasive metadata anomaly regarding sample identifiers requires further investigation to ensure correct baseline segmentation. Most critically, all 11 runs failed the noise threshold validation, with Run 32 exhibiting catastrophic instability. The presence of both linear and non-linear drift, as confirmed by statistical metrics and visual inspection in Figure 1, necessitates the implementation of robust baseline correction algorithms. Future workflows must incorporate these correction models and address the metadata inconsistencies to ensure the physical validity and accuracy of the chromatographic data.

A Appendix

A.1 A: Data Analysis Output Figures



A.2 B: Code Files

```
import pandas as pd
import os

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/baseline_integrity_report.md'

# Ensure output directory exists
os.makedirs(os.path.dirname(output_file), exist_ok=True)

# Step 1: Load data with error handling
try:
    df = pd.read_csv(input_file)
except FileNotFoundError:
    raise FileNotFoundError(f"Input_file_not_found: {input_file}")
except pd.errors.EmptyDataError:
    raise ValueError(f"Input_file_is_empty: {input_file}")

# Step 2: Strip whitespace from column names to prevent KeyError
df.columns = df.columns.str.strip()

# Step 3: Verify column integrity - Correlation between 'UV_1_280_ml' and '
volume_ml'
# Check if columns exist before calculating correlation
if 'UV_1_280_ml' in df.columns and 'volume_ml' in df.columns:
    correlation = df['UV_1_280_ml'].corr(df['volume_ml'])
    exclude_uv_ml = correlation > 0.99
else:
    correlation = None
    exclude_uv_ml = False

# Step 4: Segment data - Filter rows where 'volume_ml' < 0
baseline_df = df[df['volume_ml'] < 0]

# Calculate missingness percentages for metadata columns
baseline_missing_sample_code = baseline_df['Sample_Code'].isna().sum() / len(
    baseline_df) * 100 if len(baseline_df) > 0 else 0
baseline_missing_fraction_id = baseline_df['Fraction_unique_ID'].isna().sum() /
    len(baseline_df) * 100 if len(baseline_df) > 0 else 0

# Verify high missingness for baseline identification (Step 4 requirement)
# Confirmation: High missingness (>50%) in Sample_Code and Fraction_unique_ID
# confirms baseline/wash period
baseline_missingness_confirmed = (baseline_missing_sample_code > 50) and (
    baseline_missing_fraction_id > 50)

# Step 5: Validate baseline segment
# Logic adheres to Plan:
# 1. Signal variance must be consistent (low).
# 2. Missingness must be high (>= 50%). If missingness is low, segmentation is
    likely wrong.

baseline_valid = True
if len(baseline_df) > 0:
    # Check variance in multiple UV signal columns
    # Plan: 'consistent signal levels'. We flag as invalid if variance is
        unexpectedly high.
```

```

uv_columns = ['UV_1_280_mAU', 'UV_2_260_mAU']
existing_uv_cols = [col for col in uv_columns if col in baseline_df.columns]

if existing_uv_cols:
    variances = baseline_df[existing_uv_cols].var()
    # Placeholder: 'Consistent signal levels' implies low variance. Threshold
    # 1.0 is a conservative placeholder pending domain input for specific '
    # flat' signal definition.
    VARIANCE_THRESHOLD = 1.0
    if (variances > VARIANCE_THRESHOLD).any():
        baseline_valid = False

# Check missingness: Plan expects high missingness.
# Validation fails if missingness is NOT high (i.e., < 50%).
if baseline_missing_sample_code < 50 or baseline_missing_fraction_id < 50:
    baseline_valid = False

# Step 6: Group the filtered baseline data by 'run_no'
baseline_grouped = baseline_df.groupby('run_no').size()

# Prepare output report using vectorized string operations
report_lines = [
    "#_Baseline_Integrity_Report",
    "",
    f"Correlation_coefficient_between_'UV_1_280_ml'_and_'volume_ml':_{correlation
    :.4f}" if correlation is not None else "Correlation_coefficient_between_'
    UV_1_280_ml'_and_'volume_ml':_N/A_(Columns_missing)",
    f"Column_'UV_1_280_ml'_excluded_from_signal_analysis:_{'Yes'_if_exclude_uv_ml_
    else_'No'}",
    "",
    "Count_of_rows_identified_as_baseline_per_'run_no':",
]

# Vectorized string join for grouped data
if not baseline_grouped.empty:
    run_counts = [f"Run_{run_no}:_{count}_rows" for run_no, count in
        baseline_grouped.items()]
    report_lines.append('\n'.join(run_counts))

report_lines.extend([
    "",
    f"Summary_of_missingness_in_'Sample_Code'_for_the_baseline_region:_{
    baseline_missing_sample_code:.2f}%_(Expected~75%)",
    f"Summary_of_missingness_in_'Fraction_unique_ID'_for_the_baseline_region:_{
    baseline_missing_fraction_id:.2f}%_(Expected~75%)",
    f"Baseline_missingness_confirmation_(Expected~75%,_Actual>50%):_{'Yes'_if_
    baseline_missingness_confirmed_else_'No'}",
    f"Validation_status_of_baseline_segment:_{str(baseline_valid)}"
])

# Write report to file
with open(output_file, 'w') as f:
    f.write('\n'.join(report_lines))

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression

```

```

from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import r2_score
import os
import logging

# Configure logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s_%(levelname)s_%(message)s')
logger = logging.getLogger(__name__)

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/baseline_stability_report.md'
BASELINE_THRESHOLD = 0.0
PHYSICAL_LIMIT_LOW = -10.0
# Plan specifies 0.5% (0.005 decimal) RSD threshold
NOISE_THRESHOLD_RSD = 0.005

# Required columns
REQUIRED_COLS = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']

try:
    # Read data
    df = pd.read_csv(INPUT_FILE, usecols=REQUIRED_COLS)

    # Strip whitespace from column names
    df.columns = df.columns.str.strip()

    # Explicit existence check for all required columns
    missing_cols = [col for col in REQUIRED_COLS if col not in df.columns]
    if missing_cols:
        logger.error(f"Missing required columns: {missing_cols}. Exiting gracefully.")
        raise KeyError(f"Missing columns: {missing_cols}")

    # Filter for baseline region
    df_baseline = df[df['volume_ml'] < BASELINE_THRESHOLD].copy()

    if df_baseline.empty:
        logger.warning("No data found in baseline region (volume_ml < 0). Skipping analysis.")
        results = []
    else:
        results = []
        unique_runs = df_baseline['run_no'].unique()

        for run_id in unique_runs:
            run_data = df_baseline[df_baseline['run_no'] == run_id]

            # Defensive check: ensure run_data has required columns
            if not all(col in run_data.columns for col in REQUIRED_COLS):
                logger.warning(f"Run {run_id} missing required columns. Skipping.")
                continue

            run_result = {'Run_ID': run_id}
            physical_violation = False
            noise_violation = False

```



```

# Extract arrays
x_vol = run_data['volume_ml'].values
y_280 = run_data['UV_1_280_mAU'].values
y_260 = run_data['UV_2_260_mAU'].values

X = x_vol.reshape(-1, 1)

# Pre-compute polynomial features to ensure availability for both
wavelengths
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

# Initialize variables to 0.0 to prevent UnboundLocalError if
exceptions occur
# Removed redundant initialization of r2_diff here as it is assigned
in the loop logic
r2_diff_280 = 0.0
r2_diff_260 = 0.0
slope_280 = intercept_280 = r2_lin_280 = r2_poly_280 = rsd_280 = 0.0
slope_260 = intercept_260 = r2_lin_260 = r2_poly_260 = rsd_260 = 0.0

# Flags to track calculation success for r2_diff reporting
calc_failed_280 = False
calc_failed_260 = False

# Process UV_1_280_mAU
if np.any(y_280 < PHYSICAL_LIMIT_LOW):
    physical_violation = True

try:
    # Linear Regression 280
    model_lin_280 = LinearRegression().fit(X, y_280)
    slope_280 = model_lin_280.coef_[0]
    intercept_280 = model_lin_280.intercept_
    pred_lin_280 = model_lin_280.predict(X)
    r2_lin_280 = r2_score(y_280, pred_lin_280)

    # Polynomial Regression 280
    model_poly_280 = LinearRegression().fit(X_poly, y_280)
    pred_poly_280 = model_poly_280.predict(X_poly)
    r2_poly_280 = r2_score(y_280, pred_poly_280)

    # RSD Calculation 280
    mean_280 = np.mean(y_280)
    residuals_280 = y_280 - pred_lin_280
    std_res_280 = np.std(residuals_280, ddof=1)

    if abs(mean_280) < 1e-9:
        rsd_280 = 0.0
    else:
        rsd_280 = std_res_280 / abs(mean_280)

    if rsd_280 > NOISE_THRESHOLD_RSD:
        noise_violation = True

    # Calculate R2 difference
    r2_diff_280 = r2_poly_280 - r2_lin_280

except Exception as e:

```

```

        logger.error(f"Error processing UV_280 for run {run_id}: {e}")
        calc_failed_280 = True
        # Ensure all variables are set to defaults
        slope_280 = intercept_280 = r2_lin_280 = r2_poly_280 = rsd_280 =
            0.0
        r2_diff_280 = 0.0

# Process UV_260_mAU
if np.any(y_260 < PHYSICAL_LIMIT_LOW):
    physical_violation = True

try:
    # Linear Regression 260
    model_lin_260 = LinearRegression().fit(X, y_260)
    slope_260 = model_lin_260.coef_[0]
    intercept_260 = model_lin_260.intercept_
    pred_lin_260 = model_lin_260.predict(X)
    r2_lin_260 = r2_score(y_260, pred_lin_260)

    # Polynomial Regression 260
    model_poly_260 = LinearRegression().fit(X_poly, y_260)
    pred_poly_260 = model_poly_260.predict(X_poly)
    r2_poly_260 = r2_score(y_260, pred_poly_260)

    # RSD Calculation 260
    mean_260 = np.mean(y_260)
    residuals_260 = y_260 - pred_lin_260
    std_res_260 = np.std(residuals_260, ddof=1)

    if abs(mean_260) < 1e-9:
        rsd_260 = 0.0
    else:
        rsd_260 = std_res_260 / abs(mean_260)

    if rsd_260 > NOISE_THRESHOLD_RSD:
        noise_violation = True

    # Calculate R2 difference
    r2_diff_260 = r2_poly_260 - r2_lin_260

except Exception as e:
    logger.error(f"Error processing UV_260 for run {run_id}: {e}")
    calc_failed_260 = True
    # Ensure all variables are set to defaults
    slope_260 = intercept_260 = r2_lin_260 = r2_poly_260 = rsd_260 =
        0.0
    r2_diff_260 = 0.0

# Calculate final R2 difference
# If both failed, r2_diff remains 0.0. If one failed, max takes the
# valid one.
# If both succeeded, max takes the larger improvement.
r2_diff = max(r2_diff_280, r2_diff_260)

# Determine Non_Linear_Fit_Improvement value with failure flag
# Refactored to rely exclusively on failure flags to avoid ambiguity
if calc_failed_280 or calc_failed_260:
    non_linear_val = "Calculation Failed"
else:

```

```

        non_linear_val = f"{r2_diff:.6f}"

    run_result.update({
        'UV_280_Slope': f"{slope_280:.6f}",
        'UV_280_Offset': f"{intercept_280:.6f}",
        'UV_280_RSD': f"{rsd_280:.4f}",
        'UV_260_Slope': f"{slope_260:.6f}",
        'UV_260_Offset': f"{intercept_260:.6f}",
        'UV_260_RSD': f"{rsd_260:.4f}",
        'Non_Linear_Fit_Improvement': non_linear_val,
        'Physical_Limit_Violation': str(physical_violation),
        'Noise_Threshold_Violation': str(noise_violation)
    })

    results.append(run_result)

# Generate Output
output_df = pd.DataFrame(results)

os.makedirs('/workspace', exist_ok=True)

# Use 'w' mode to overwrite previous report, ensuring clean output
with open(OUTPUT_FILE, 'w') as f:
    f.write("#_Baseline_Stability_Report\n\n")
    f.write("##_Step2:_Run-Specific_Baseline_Stability_&_Drift_Profiling\n\n")
    )

    # Check line count constraint: Header + Separator + Rows <= 20
    # Header lines: 1 (Title) + 1 (Subtitle) + 1 (Empty) + 1 (Table Header) +
    # 1 (Separator) = 5 lines
    # Max rows allowed = 20 - 5 = 15 rows.
    # If results > 15, truncate or aggregate. Here we truncate to first 15
    runs.
    max_rows = 15
    if len(output_df) > max_rows:
        logger.warning(f"Output_exceeds_20-line_limit._Truncating_to_first_{
            max_rows}_runs.")
        output_df = output_df.head(max_rows)

    if not output_df.empty:
        headers = output_df.columns.tolist()
        f.write("|_ " + "|_".join(headers) + "|_\\n")
        f.write("|_ " + "|_".join(["---"] * len(headers)) + "|_\\n")

        # Optimized iteration: convert to list of lists for faster string
        # joining
        rows = output_df.values.tolist()
        for row in rows:
            row_str = "|_".join([str(val) for val in row])
            f.write(f"|_{row_str}|_\\n")
        else:
            f.write("|_No_Data_Available_|_\\n")

    logger.info("Baseline_stability_report_generated.")

except Exception as e:
    logger.error(f"Critical_error_during_execution:_{e}")
    raise

```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import sys

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Step 1: Determine subset of runs
# Select unique runs and sort by run_no
unique_runs = df['run_no'].unique()
unique_runs_sorted = sorted(unique_runs)

# If total runs > 5, select first 5
if len(unique_runs_sorted) > 5:
    selected_runs = unique_runs_sorted[:5]
else:
    selected_runs = unique_runs_sorted

# Step 2: Filter runs with sufficient baseline data (< 10 points in region < 0)
# Optimization: Pre-compute baseline data for all valid runs outside the loop
selected_runs_df = df[df['run_no'].isin(selected_runs)]
valid_runs = []
for run in selected_runs:
    baseline_data = selected_runs_df[(selected_runs_df['run_no'] == run) & (
        selected_runs_df['volume_ml'] < 0)]
    if len(baseline_data) >= 10:
        valid_runs.append(run)

# If no valid runs, exit early gracefully
if not valid_runs:
    print("No runs with sufficient baseline data found.")
    sys.exit(0)

# Prepare plotting
fig, axes = plt.subplots(len(valid_runs), 1, figsize=(10, 4 * len(valid_runs)))
if len(valid_runs) == 1:
    axes = [axes]

for idx, run in enumerate(valid_runs):
    # Use pre-filtered dataframe and boolean indexing
    baseline_data = selected_runs_df[(selected_runs_df['run_no'] == run) & (
        selected_runs_df['volume_ml'] < 0)].sort_values('volume_ml')

    if baseline_data.empty:
        continue

    x = baseline_data['volume_ml'].values.reshape(-1, 1)
    y1 = baseline_data['UV_1_280_mAU'].values
    y2 = baseline_data['UV_2_260_mAU'].values

    # Linear regression for y1
```

```

lin_reg_1 = LinearRegression()
lin_reg_1.fit(x, y1)
y1_lin_pred = lin_reg_1.predict(x)

# Polynomial regression (2nd order) for y1
poly = PolynomialFeatures(degree=2)
x_poly = poly.fit_transform(x)
poly_reg_1 = LinearRegression()
poly_reg_1.fit(x_poly, y1)
y1_poly_pred = poly_reg_1.predict(x_poly)

# Linear regression for y2 (UV_2_260_mAU)
lin_reg_2 = LinearRegression()
lin_reg_2.fit(x, y2)
y2_lin_pred = lin_reg_2.predict(x)

# Polynomial regression (2nd order) for y2
poly_reg_2 = LinearRegression()
poly_reg_2.fit(x_poly, y2)
y2_poly_pred = poly_reg_2.predict(x_poly)

# Plotting
ax = axes[idx]
ax.plot(x, y1, label='Raw□Data□(UV_1_280)', color='blue', alpha=0.7)
ax.plot(x, y2, label='Raw□Data□(UV_2_260)', color='green', alpha=0.7)

# Plot fits for y1
ax.plot(x, y1_lin_pred, label='Linear□Fit□(UV_1)', color='red', linestyle='--',
        )
ax.plot(x, y1_poly_pred, label='Poly□Fit□(UV_1)', color='orange', linestyle='-.')

# Plot fits for y2
ax.plot(x, y2_lin_pred, label='Linear□Fit□(UV_2)', color='purple', linestyle='--')
ax.plot(x, y2_poly_pred, label='Poly□Fit□(UV_2)', color='brown', linestyle='-.')

# Reference lines
ax.axhline(0, color='black', linewidth=1)
ax.axhline(-10, color='gray', linestyle=':', linewidth=1, label='Physical□
    Limit□(-10□mAU)')

# Shade region below -10
ax.axhspan(-np.inf, -10, color='gray', alpha=0.2)

ax.set_title(f'Run□{run}')
ax.set_xlabel('Volume□(mL)')
ax.set_ylabel('Absorbance□(mAU)')
ax.legend(loc='best')
ax.grid(True)

plt.suptitle('Pre-Injection□Baseline□Drift□&□Non-Linearity□by□Run□(First□5□Runs)',
            fontsize=14)
plt.tight_layout()
plt.savefig('/workspace/baseline_drift_visualization.png', dpi=300)
plt.close()

```

Listing 3: Code_3